PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Contents

1. Project Overview 3
2. Background & Business Objective 4
3. Product Purpose 5
4. System Functionalities 6
5. User Stories & Use Cases 10
6. Features Included (Scope Definition) 12
7. Features Not Included (Scope Control) 13
8. Assumptions & Constraints 14
9. Risks & Questions Throughout Development 14

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: Review2Revenue

Version: 1.0

1. Project Overview

- **Problem Statement:** In the highly competitive F&B industry, valuable customer feedback is fragmented and buried within large volumes of unstructured review data. Small and medium-sized restaurant owners lack the time and analytical tools to manually extract actionable operational insights, often resulting in guesswork rather than data-driven business decisions.
- **Target Domain:**
  - Diners (Consumers): Empowered to discover ideal dining spots through hyper-personalized recommendations backed by AI-generated justifications. They also contribute to the ecosystem by submitting post-visit reviews, creating a continuous feedback loop.
  - Vendors (Restaurant Owners): Equipped with a decision intelligence dashboard that instantly transforms fragmented, unstructured customer reviews into quantified, actionable operational strategies.
- **Proposed Solution Summary:** Review2Revenue is a dual-sided, AI-powered decision intelligence platform that bridges the gap between customer feedback and F&B operations. Powered by advanced language models, it acts as an intelligent concierge for diners, providing highly personalized restaurant recommendations with clear, logical justifications. Simultaneously, it serves as an automated operational consultant for vendors, ingesting unstructured review data to identify core business bottlenecks. Instead of simple sentiment analysis, our system generates quantified, actionable strategies such as staffing adjustments or menu pricing changes which enable restaurant owners to make confident, data-driven decisions that directly drive revenue growth.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Background & Business Objective

- **Background of the Problem:** In today's data-heavy Food and Beverage industry, a critical disconnect exists where restaurant owners struggle to extract actionable insights from fragmented reviews. They often rely on guesswork instead of data to improve their business. On the other hand, diners face decision fatigue trying to find personalized, high-value meals beyond generic star ratings.
- **Importance of Solving This Issue:** For F&B vendors, it eliminates operational guesswork, empowering them to rapidly implement strategic changes such as dynamic pricing adjustments or targeted service improvements that directly drive monthly revenue growth. Simultaneously, it democratizes the dining experience for consumers by ensuring they can consistently discover high-quality and reasonably priced meals that are backed by authentic as well as AI-analyzed community feedback.
- **Strategic Fit / Impact:** The system integrates Z.AI's General Language Model (GLM). Z.AI acts as the central reasoning engine which bridges the gap between raw data and actionable intelligence to create immediate value for both market segments.

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Product Purpose

3.1. Main Goal of the System: To automatically transform massive volumes of unstructured customer feedback into quantified and executable business strategies for restaurant operators in order for them to drive revenue growth. Furthermore, it simultaneously delivers hyper-personalized, transparent and value-optimized dining recommendations for consumers.

3.2. Intended Users (Target Audience):

- SME Restaurant and Cafe Owners
- Restaurant Operations Managers
- New F&B Entrepreneurs
- Value-Conscious Diners
- Tourists and Travelers

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. System Functionalities

4.1. Description: The system is engineered as a comprehensive and AI-powered decision intelligence ecosystem that operates through a dual-role architecture which are Vendor and Diner. The system leverages Z.AI's advanced Large Language Models as its core to ingest, process and interpret various amounts of unstructured customer rating and review data stored in a PostgreSQL database. The system ensures a seamless and intuitive user experience by cleanly separating the heavy data processing and AI reasoning in the backend from the user interface.

4.2. Key Functionalities:

Diners	Vendors
Conversational Discovery ● Use natural language prompts to describe their specific dining needs, context, or dietary restrictions.	**Performance Overview Dashboard** ● A centralized visual interface that automatically synthesizes hundreds of raw customer reviews into an easy-to-read, real-time sentiment score and immediately highlights the critical problems.
Justified Recommendations ● Generate "Top Picks Restaurant" by scanning extensive backend review data. It provides clear, logical reasons explaining exactly why each restaurant fits the user's specific request.	**Decision Impact Simulator (What-If Analysis)** ● Features interactive UI sliders that allow vendors to simulate potential business changes (like raising menu prices) to instantly view AI-generated projections on how those changes will affect future sentiment and revenue.
Dynamic Trade-Off Comparison ● Automatically evaluates shortlisted restaurants against each other to highlight specific pros and cons; for example, food quality, service and price. This allows diners to make informed compromises based on what	**Actionable & Quantified Insights** ● Translates vague customer complaints into specific, data-backed operational directives (e.g., adjusting staff during rush hours) complete with predicted revenue impacts.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Diners	Vendors
matters most to them.	
Rating Breakdown ● A summarized rating(out of 5) will be given by the system for each recommendation provided in the "Top Pick Restaurant".	**On-Demand Competitor Analysis** ● Allows vendors to input external reviews of rival restaurants to instantly analyze competitor weaknesses and discover profitable local market gaps without needing internal database access.

4.3. AI Model & Prompt Design

4.3.1. Model Selection:

- The model selected is Z.AI's General Language Model(GLM) for its context-aware reasoning and prescriptive analytics. For example, it can deduce underlying operational bottlenecks from fragmented complaints. GLM also excels at adhering to complex instructions which enables it to output structured data such as simulated revenue impacts and specific rating metrics.

4.3.2. Prompting Strategy:

- The selected strategy is Few-Shot Prompting because it perfectly balances reasoning accuracy, strict structural compliance and system speed. By embedding specific input-output examples into the prompt, we force the AI to return perfectly formatted JSON data which is critical for our React frontend to seamlessly render the decision simulator and charts without backend parsing errors. Furthermore, these examples calibrate the model's business logic, teaching it exactly how to translate vague customer complaints into professional and quantified operational directives rather than generic advice. Few-shot prompting delivers this complex, high-quality analysis in a single API call, ensuring the low latency necessary for a highly responsive and real-time user experience during the live demo.

© UMHackathon 2026

4.3.3. **Context & Input Handling**:

- To manage the vast amounts of unstructured customer feedback, the system persistently stores all historical review data within a Supabase PostgreSQL database to ensure complete data retention. During active analysis, the FastAPI backend constructs structured few-shot prompts tailored to the active Large Language Model. For the primary AI engine, the system leverages Z.AI's massive context window of up to 5,000,000 tokens, allowing it to ingest and analyze comprehensive restaurant histories without requiring aggressive text truncation. When the system routes requests to secondary fallback models, the input size is dynamically managed to adhere strictly to Groq's specific token constraints. Furthermore, for on-demand analysis of competitors or unlisted restaurants, vendors can upload external review datasets via CSV. To maintain system stability and rapid response times, this external data ingestion features a strict hard limit of exactly 500 reviews per file. If a user attempts to upload a CSV exceeding this 500-review threshold, the application actively rejects the input at the upload stage, entirely preventing the file transfer and prompting the user to refine and reduce their dataset before proceeding.

© UMHackathon 2026
Email: umhackathon@um.edu.my

4.3.4. **Fallback & Failure Behavior:**

- To guarantee frontend stability, the architecture employs a multi-layered error-handling system that uses Pydantic validation to immediately intercept malformed outputs and block AI hallucinations. If the primary Z.AI engine fails or returns unusable data, the system seamlessly cascades to secondary LLM providers like Groq or Gemini. In the event of a total external API outage, the backend automatically falls back to a locally trained sentiment classifier or a keyword-based database summary. This ensures the user still receives actionable metrics alongside a graceful degradation message, completely preventing a hard crash.

© UMHackathon 2026

5. User Stories & Use Cases

- **User Stories:**
  "As a busy SME cafe owner, I want to view an AI-summarized dashboard of my most recurring customer complaints, so that I can instantly identify my operational bottlenecks without spending hours manually reading hundreds of Google reviews."

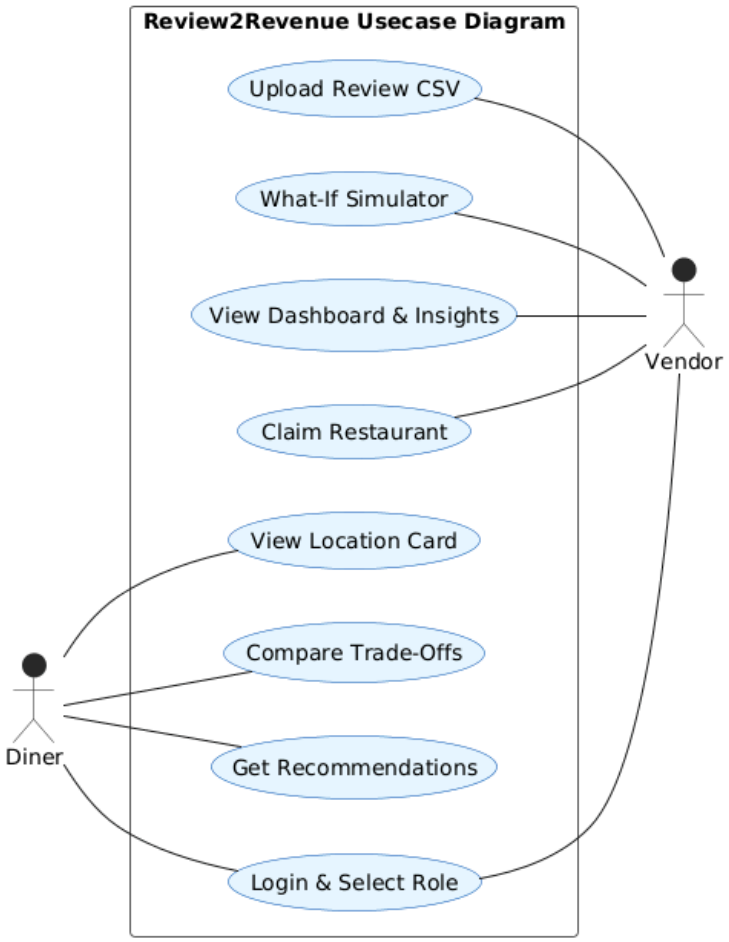
- **Use Cases (Main Interactions):**

  Prompt to UI Flow: Vendors / Diners enter the prompt ➔ UI displays a seamless loading state ➔ Review2Revenue analyzes the request ➔ UI dynamically renders the final interface

  Flow Generation: React frontend sends a request ➔ FastAPI strictly validates the payload via Pydantic ➔ The API router splits the logic based on the user's role (Diner or Vendor) ➔ The database module queries PostgreSQL to fetch pre-computed metrics or aggregate relevant unstructured reviews ➔ FastAPI injects this database context into role-specific, few-shot prompt templates ➔ executes the inference using a Multi-Provider Cascading Fallback mechanism (Groq ➔ Gemini ➔ Z.AI) ➔ The AI engine outputs a strict, structured JSON payload ➔ FastAPI returns this payload to React to instantly render the Dashboard or Recommendations.

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Use Case Diagram:**



© UMHackathon 2026

Email: umhackathon@um.edu.my

6. Features Included (Scope Definition)

- Conversational Search Interface, which allows diners to use natural language to request restaurants based on highly specific contexts, dietary needs, or vibes.
- Dynamic Trade-Off Comparison, which evaluates top restaurant picks by highlighting specific pros and cons to aid decision-making.
- Essential vendor contact information such as phone numbers, addresses, and operating hours will be displayed by the time diners confirm their preferable restaurant.
- • The Performance Overview Dashboard provides a centralized and visual summary of current overall sentiment scores and top recurring customer pain points to the vendors.
- Interactive UI sliders allow vendors to manipulate variables such as menu pricing or portion sizes in order to view real-time AI projections on rating changes and revenue.
- A high-availability AI routing mechanism that automatically switches between Groq, Gemini and Z.AI GLM to prevent system failure during API downtimes or rate limits.
- Pre-Computed Metrics Storage where database architecture that stores baseline metrics for rapid retrieval, minimizing LLM token usage and processing latency.

© UMHackathon 2026

Email: umhackathon@um.edu.my

7. Features Not Included (Scope Control)

- The platform will not connect directly to a restaurant's internal Point of Sale (POS) or Customer Relationship Management (CRM) systems.
- The platform itself is strictly a discovery and decision-intelligence engine, while the system provides contact details for diners to make their own reservations. It will not feature native transactional capabilities such as an automated in-app booking system, API calendar syncing with restaurants, payment processing, or food delivery routing.
- The deliverable is a responsive web application built with React. The development of native iOS or Android applications is excluded from this phase.
- The system will not scrape live reviews from Google Maps in real-time. The Minimum Viable Product (MVP) relies on a pre-cleaned and static dataset loaded into our database via CSV to demonstrate the AI reasoning engine.

© UMHackathon 2026

Email: umhackathon@um.edu.my

8. Assumptions & Constraints

- **LLM Cost Constraint:** Pre-computing and filtering metrics in the database drastically minimizes expensive AI token usage during live inference.
- **Technical Constraints:** The system relies on a static as well as pre-cleaned review database because strict third-party API limits prevent real-time web scraping.
- **Performance Constraints:** Inherent AI processing latency (approximately 3-8 seconds) necessitates the use of dynamic frontend loading states to maintain a smooth user experience.
- **User Input:** The platform definitely acts as a decision-support tool requiring manual human execution, rather than an autonomous agent that directly alters business operations.

9. Risks & Questions Throughout Development

- **Design Similarity:** How are we ensuring that the AI is not generating identical restaurant recommendations for every diner or spitting out the exact same generic advice to every vendor?
- **Frame Stability:** How are we ensuring the stability of the AI's analysis so that a vendor's sentiment score or recommended revenue strategy doesn't wildly fluctuate between two identical user sessions when the underlying database hasn't changed?
- **Drawing Interpretation**: How are we ensuring the AI accurately interprets unstructured, messy customer reviews, especially those containing local slang (*Bahasa Rojak*), typos, or sarcasm without misclassifying a positive vibe as a negative complaint?

© UMHackathon 2026

Email: umhackathon@um.edu.my